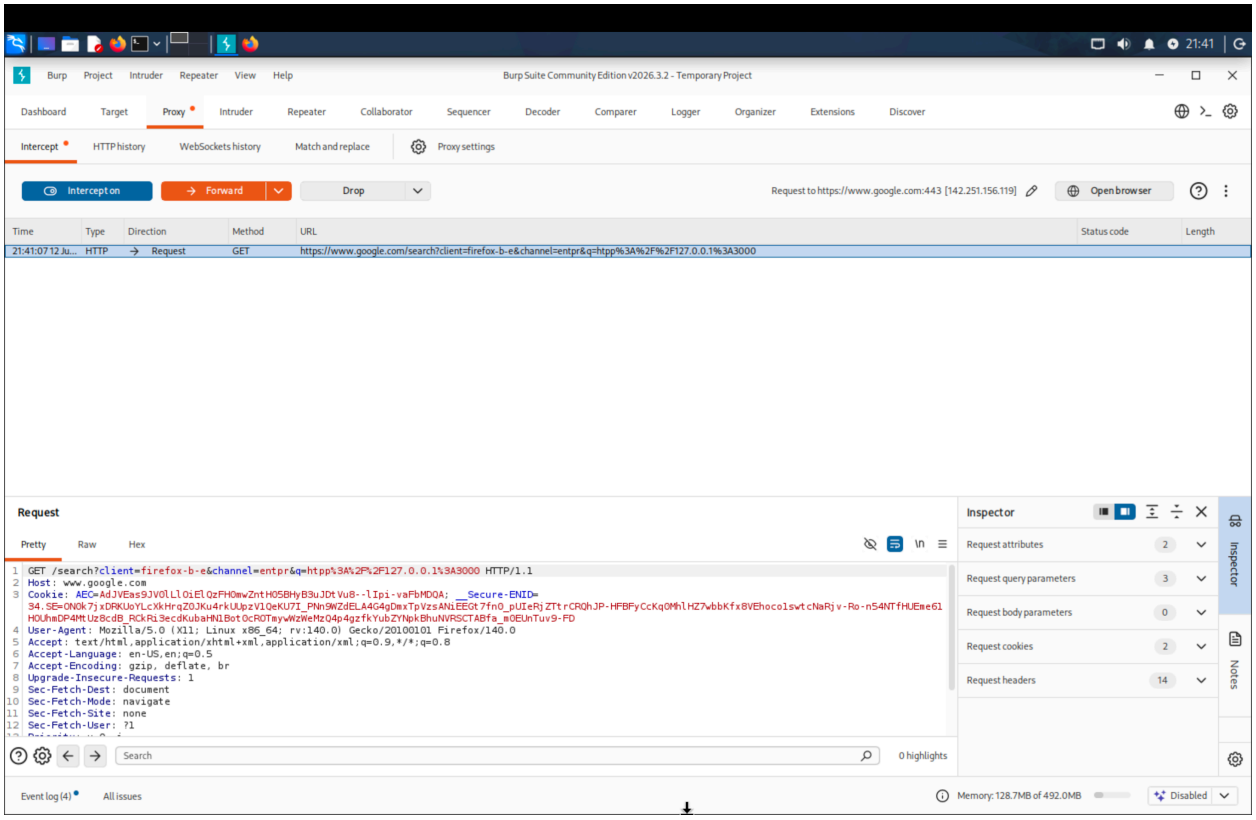
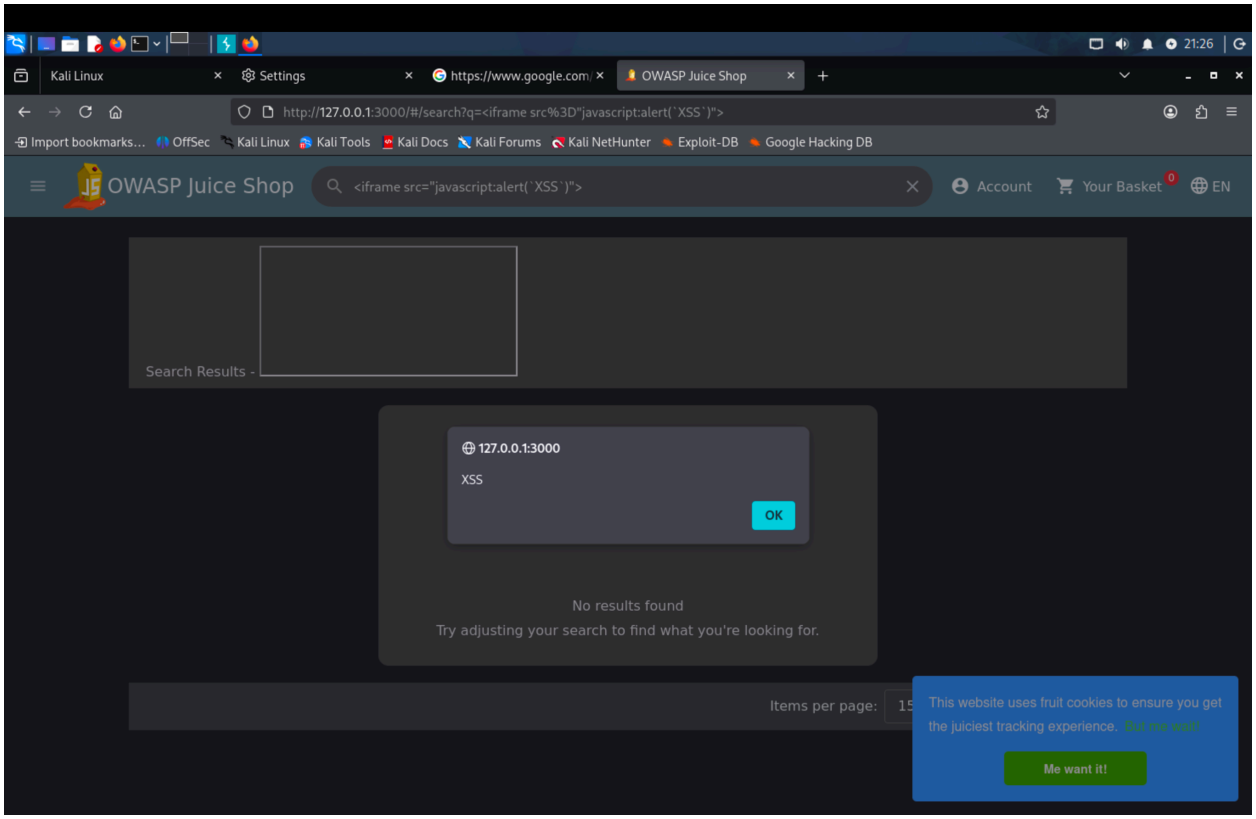
Name; Rofiat Akinremi
Lab Environment: Kali,Burpsuite, Juice Shop.
Date:12th of July,2026

SQL Injection (Authentication Bypass)I used the Firefox Burpsuit community



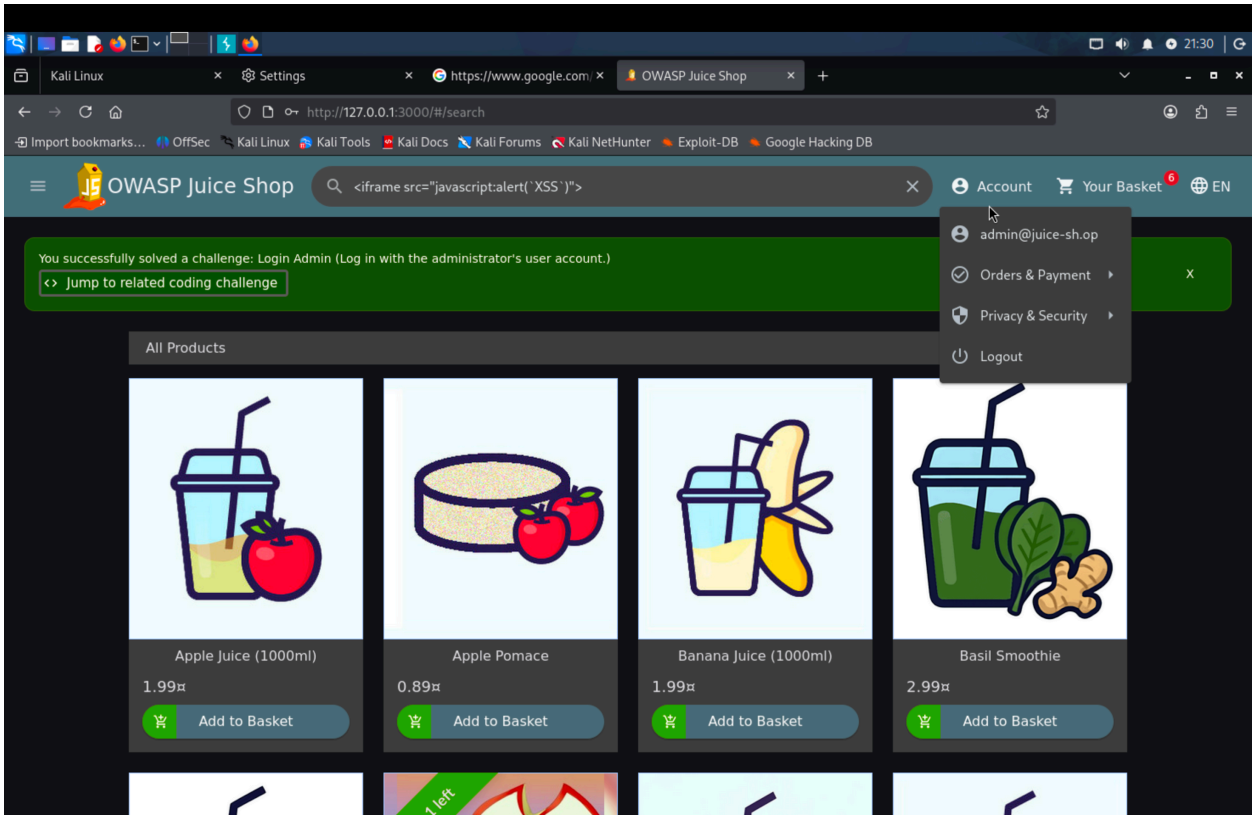
By entering a script tag into the search field, I forced the browser to execute a pop up alert. The application failed to treat my input as harmless text, instead running it as active code.I used the ; “<iframe src="javascript:alert XSS)">”. And this is my Proof of Concept for Reflected XSS

demonstrating code execution via search input.



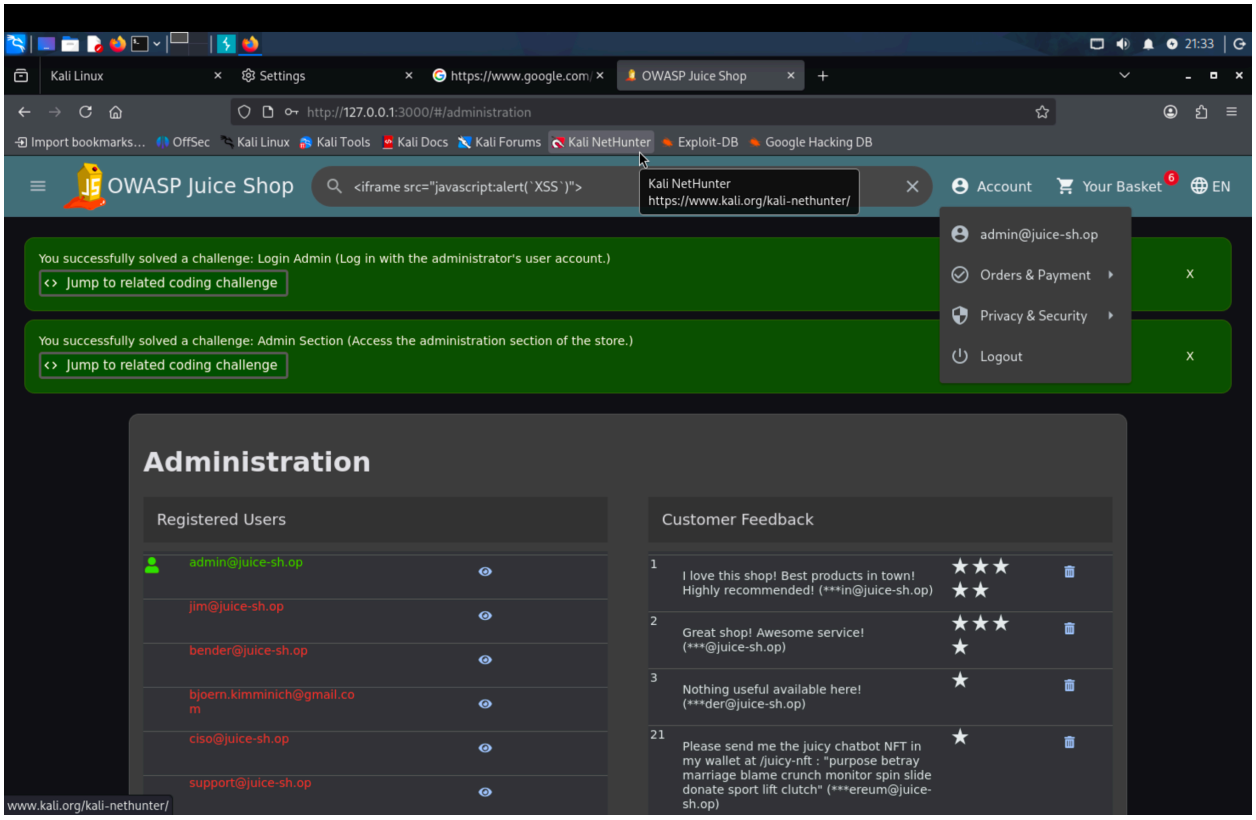
Reflected Cross Site Scripting (XSS)I tested the login form by entering ' OR 1=1--. This tricked the backend database into believing the authentication condition was "true," allowing me

to bypass the login portal and enter the system as the administrator without a password.



Administrative Interface Exposure (Broken Access Control): The administrative panel was hidden but not protected so all I had to do was manually navigating to the “/administration” URL path, I bypassed the user interface restrictions,proving that the application lacked proper server

side permission checks.



Vulnerability Analysis and Impact::

Vulnerability	**C / I / A Impact**	**Severity**	**Real-World Scenario Risk**
SQL Injection (Auth Bypass)	C: Critical / I: Critical / A: High	**Critical**	Full account takeover by the attacker and administrative exfiltration of financial records.
Reflected XSS	**C: High / I: High / A: Low**	**High**	Phishing link steals active user session tokens via malicious scripts.

Broken Access Control	C: High / I: High / A: Medium	**High**	Unauthenticated users scale privileges from Guest to Administrator.

Remediation and Defense

SQL Injection: Implement parameterized queries (prepared statements). Enforce least-privilege database user permissions to prevent deletion or structural modification.

XSS: Implement robust input sanitization and contextual output encoding (converting < and > to safe HTML entities). Deploy a strict Content Security Policy (CSP) header.

Broken Access Control: Enforce strict server-side role validation via session tokens for every API endpoint. Protect session management using HttpOnly and Secure cookie flags over HTTPS.

Reflection; During this security assessment, I found the Broken Access Control vulnerability to be the most challenging to exploit. Unlike SQL Injection or XSS which rely on injecting malicious payloads into input fields, identifying an access control flaw required me to shift my mindset from what can I inject to how is this application architected. It forced me to move beyond the user interface to explore the application's hidden logic proving that relying on "security through obscurity" like simply hiding a link is a dangerous practice that fails immediately when a attacker start manual path discovery. This project taught me that offensive and defensive security are two sides of the same coin. I realized that automated tools and manual testing are most powerful when used together. However, these tools often miss logical flaws. Manual testing provides the human creativity and contextual understanding needed to uncover deep seated issues that logic based scanners simply cannot comprehend. In a real world fintech environment, a defense in depth strategy must combine the efficiency of automation with the precision of manual penetration testing to ensure that sensitive financial data remains fully protected.

References

OWASP Foundation. (2026). SQL Injection Prevention Cheat Sheet.

OWASP Foundation. (2026). Cross-Site Scripting (XSS) Prevention Cheat Sheet.

OWASP Foundation. (2026). Authorization Cheat Sheet.

OWASP Foundation. (2021). OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks.